pdf_downloading: root cause of zero-download windows

Investigation of recurring 15-minute buckets with zero completed downloads, 2026-07-13 to 2026-07-19

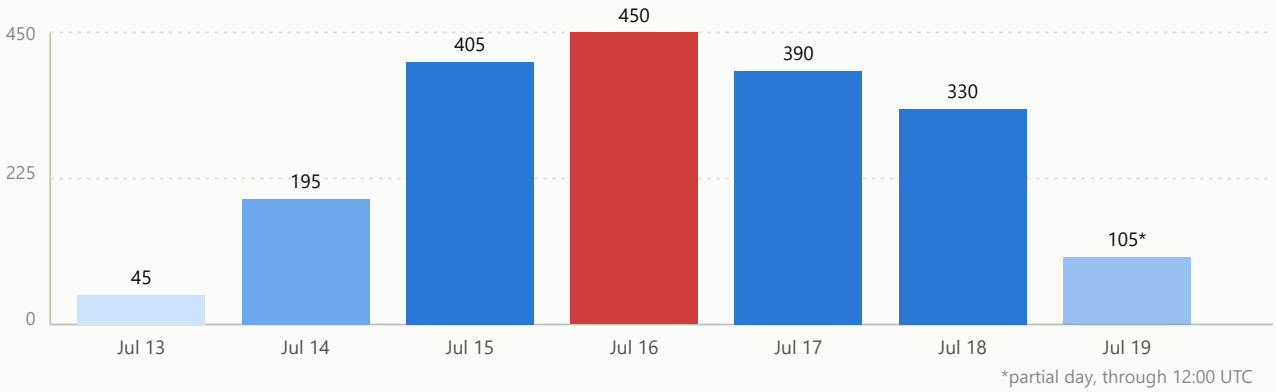
Prepared 2026-07-19 · data source: TextCorpuses.dbo.PdfDocuments (ClaimedAt), Airflow dag_run/task logs · corpus-host (172.21.128.103)

> **Bottom line:** the dominant remaining cause of zero-download 15-minute windows since 2026-07-17 is a concurrency bug in `pdf-downloading-dag.py` — a single slow-but-not-hung download can block the entire task's completion (and therefore the next batch's claims) for 50–100+ minutes, far beyond the 5-minute per-download timeout the code appears to enforce. This is confirmed with direct log evidence from two separate incidents. A one-line fix has been implemented and **deployed to production** (2026-07-19, ~14:05 UTC) — it takes effect starting with the next `pdf_downloading` run.

1. How “zero downloads” is measured

The ops report buckets `PdfDocuments.ClaimedAt` into 15-minute windows. Tracing the stored procedure that sets it (`GetPdfToDownload`), **ClaimedAt is stamped the moment a URL is handed to a worker**, not when a download succeeds — so a zero bucket means literally *no worker claimed any URL at all* in that window, i.e. the whole task was idle, not merely failing. (The comment in `generate_ops_report.py` describing ClaimedAt as “set only on a successful download” is inaccurate and should be corrected — see recommendations.)

Zero-download minutes per day (out of 1440 possible)



Jul 16 stands out because it falls inside the already-documented FTP passive-mode outage (fixed the night of Jul 16→17 — see project history). Jul 15 overlaps the disk-I/O bottleneck that was resolved by the NVMe migration. Both of those are known, closed issues and are **not** the subject of this report. The focus here is what's still causing gaps *after* those fixes — Jul 17, 18 and 19 still show 330–390 zero-minutes per day, and that remainder traces to a single new mechanism.

2. Primary root cause (confirmed): the executor's own shutdown re-introduces the hang it was built to bound

CONFIRMED — REPRODUCED TWICE HIGH IMPACT

On 2026-07-17, `pdf-downloading-dag.py` was fixed so a hung download couldn't block the batch forever: each of the 500 per-run downloads is wrapped in `future.result(timeout=300)`, and a straggler past 5 minutes is logged and treated as unclaimed so the collection loop can move on. That fix is real and necessary, but it only bounds how long the *main thread waits for a result value* — it does not stop the underlying worker **thread** from continuing to run in the background.

The code still opens the executor as `with ThreadPoolExecutor(...) as executor:`. When that block exits, Python's context manager calls `executor.shutdown(wait=True)` — which blocks until *every* submitted thread has actually returned, including ones the loop already gave up on. So a single download that is slow but not truly hung (e.g. a large file trickling through a barely-alive proxy, never tripping the 30-second per-chunk read timeout because *some* bytes keep arriving) holds up the entire task's completion — and with it, the next `@continuous` batch's claims — for as long as that one download actually takes, regardless of the 5-minute timeout.

Evidence

RUN (START → END)	DURATION	LAST "HUNG PAST 300S" LOG LINE	FOLLOWING ACTIVITY	MATCHING ZERO-DOWNLOAD BUCKET
2026-07-18 14:51:48 → 17:22:17	150 min	15:40:10	**Total silence** from 15:40:10 to 17:22:17 (102 min), then immediately `Done.`	15:30–17:00 (105 min)
2026-07-19 09:54:57 → 11:21:49	87 min	10:31:32	**Total silence** from 10:31:32 to 11:21:48 (50 min), then immediately `Done.`	10:30–11:00 (45 min)

In both cases the pattern is identical: the last time the code logs giving up on a straggler, followed by complete silence — no proxy attempts, no errors, nothing — until the task exits. That silence is the `with` block waiting on the abandoned thread(s) to actually finish.

The same signature (anomalously long run duration directly overlapping a multi-bucket gap) also matches several other recent runs not traced down to individual log lines — e.g. the 109.8-minute run on Jul 18 00:12–02:02 (matching the 00:30–01:45 gap) and the 96.5-minute run on Jul 17 13:19–14:55 (matching the 13:45–14:30 gap). Typical healthy runs finish in 20–50 minutes; every run above ~85 minutes since Jul 17 lines up with a reported gap.

Fix DEPLOYED 2026-07-19

Stop joining stragglers before the task returns — let them finish in the background instead:

```
- with ThreadPoolExecutor(max_workers=CONCURRENCY) as executor:
-     futures = [executor.submit(downloadOne) for _ in range(TOTAL_URLS)]
-     results = []
-     for future in futures:
-         try:
-             results.append(future.result(timeout=PER_FUTURE_TIMEOUT_SECONDS))
-         except TimeoutError:
-             results.append(False)
+ executor = ThreadPoolExecutor(max_workers=CONCURRENCY)
+ futures = [executor.submit(downloadOne) for _ in range(TOTAL_URLS)]
+ results = []
+ for future in futures:
+     try:
+         results.append(future.result(timeout=PER_FUTURE_TIMEOUT_SECONDS))
+     except TimeoutError:
+         results.append(False)
+ executor.shutdown(wait=False)
```

Why this is safe: the abandoned thread keeps running until its slow download actually finishes (it isn't truly hung in these observed cases — it does eventually complete), it just no longer blocks the task or the next batch. Worst case if a thread genuinely never returns: it leaks for the life of the worker process, same as today, but bounded at the process level by the existing 3-hour `execution_timeout`. File: `dags/pdf-downloading-dag.py`.

3. Contributing factors worth fixing alongside this

3.1 Loading full response bodies into memory

NOT FULLY ROOT-CAUSED

`downloadOne()` uses `io.BytesIO(response.content)`, which buffers the entire file in RAM before it touches disk. With up to 64 concurrent downloads and some sources known to include multi-GB PDFs, this is a plausible contributor to two unexplained `failed` dag runs found in the same window (2026-07-18 23:46 and 2026-07-19 00:01, contributing to the 00:30–01:15 gap) — both logs end abruptly mid-transfer with no Python traceback, which is the signature of the process being killed externally (e.g. by the OS under memory pressure) rather than crashing on its own. This wasn't confirmed against host memory metrics for the exact failure window, so treat it as a lead, not a proven cause.

Suggested improvement: stream to a temp file (`requests.get(..., stream=True)` + chunked `iter_content()` writes) instead of materializing the whole file in memory, and stream that file to FTP rather than round-tripping through a `BytesIO`.

3.2 The ops report doesn't distinguish a real gap from an intentional one

BY DESIGN, BUT UNDER-LABELED

When every one of the 500 slots in a batch finds nothing to claim, the task deliberately sleeps 15 minutes (`QUEUE_EMPTY_BACKOFF_SECONDS`) before letting `@continuous` retrigger — this is intentional backpressure, not a bug, and produces a legitimate single zero-bucket when the backlog across every source is genuinely empty. Right now the ops report's alert ("Разрыв в загрузках PDF") fires identically for this case and for the 90+ minute architectural gaps described above, which makes the alert noisy and harder to act on.

Suggested improvement: in `generate_ops_report.py`, cross-reference each zero bucket against `dag_run` duration for the overlapping `pdf_downloading` run — a ~15-16 minute run overlapping the gap is the benign queue-empty case; anything overlapping an 80+ minute run is the bug in §2. Label them differently in the report.

3.3 Two unexplained failed runs

NEEDS FOLLOW-UP

`scheduled__2026-07-18T23:46:20` and the run immediately after it both failed with no exception recorded in the task log — activity just stops mid-batch. This is consistent with an external kill (OOM or a Celery worker restart) but wasn't confirmed against host-level memory/OOM logs in this pass. Worth a follow-up check of `docker inspect` OOM flags and host `dmesg` around future recurrences, since dmesg wasn't accessible with the current permissions during this investigation.

4. Already-fixed, not the subject of this report

ISSUE	WINDOW AFFECTED	STATUS
FTP passive-mode outage (4 stacked bugs: session leaks, unbounded thread hang, firewall port range, PASV external-IP advertisement to local peers)	Jul 16 evening – Jul 17 early morning	`Fixed, verified (19/20 statistical PASV test)`

Disk I/O saturation on `sda`	Through Jul 15/16	Fixed via NVMe migration
Proxy pool determinism (“champion” proxy reused by every worker)	Through Jul 14	Fixed (randomized top-20 selection)
Airflow `core.parallelism` ceiling	Through Jul 14/15	Fixed (32 → 64)

These are included only so the daily chart in §1 isn't misread as one ongoing problem — Jul 15/16's high totals are these, not the §2 bug.

5. Recommendations, in priority order

1. **§2 fix — done.** Deployed to production 2026-07-19. Watch the next few days of the ops report to confirm it removes the majority of remaining zero-download windows.
2. **Stream downloads instead of buffering in memory** (§3.1) — cheap change, removes a plausible OOM contributor.
3. **Teach the ops report to distinguish benign queue-empty backoffs from real stalls** (§3.2) — improves signal, no pipeline behavior change.
4. **Correct the stale comment** in `generate_ops_report.py` describing `ClaimedAt` as success-only (§1) — low effort, prevents future misdiagnosis.
5. **Watch for OOM signatures** on the next unexplained failed run (§3.3) — add a one-line check (`docker inspect --format '{{.State.OOMKilled}}'`) to the existing on-call/debugging playbook.

Data: `TextCorpuses.dbo.PdfDocuments` (ClaimedAt, 6-day window), Airflow `dag_run`/task-instance history via Postgres, and raw task logs for the two runs cited in §2. Investigation followed a root-cause-first process: reproduced the gap pattern against real dag_run durations before forming or testing any fix hypothesis.